

Short-Form Indexed Memory Architecture (SIMA)

A model-agnostic architecture for scalable long-term AI memory retrieval.

Status: Experimental / Open Research

License: TBD

Version: 0.1.0

Overview

Short-Form Indexed Memory Architecture (SIMA) is an experimental memory architecture designed to improve the efficiency of long-term memory retrieval in AI systems.

The central hypothesis is simple:

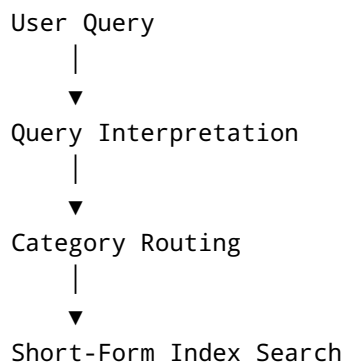
A memory system should not need to retrieve the full representation of every potentially relevant memory in order to determine which memories are actually relevant.

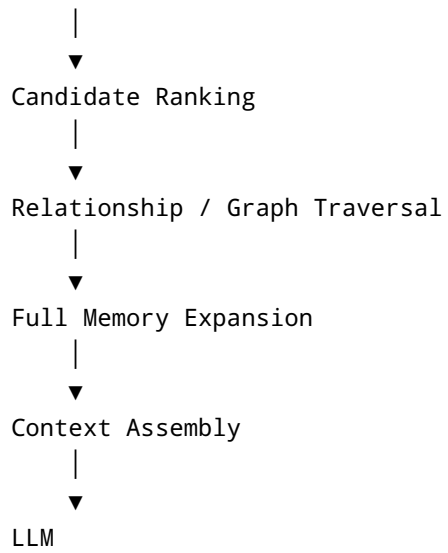
SIMA separates:

1. **Persistent memory storage**
2. **Memory retrieval representation**
3. **Memory indexing**
4. **Memory ranking**
5. **Full memory expansion**
6. **Context assembly**

Instead of searching and loading full memory objects immediately, SIMA first searches compact, structured representations and progressively expands only the memories that are likely to be relevant.

Core retrieval pipeline





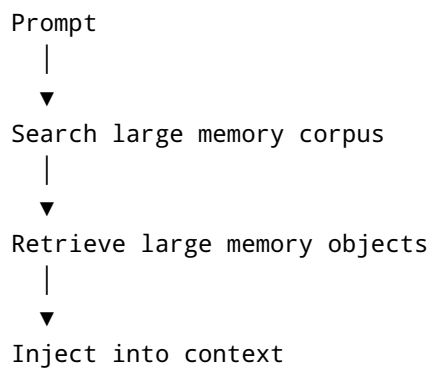
The goal is to maximize:

$$\frac{\text{Relevant Information Retrieved}}{\text{Tokens} + \text{Compute} + \text{Latency}}$$

Why SIMA?

As AI systems accumulate long-term memory, the total memory corpus can grow far beyond the model's active context window.

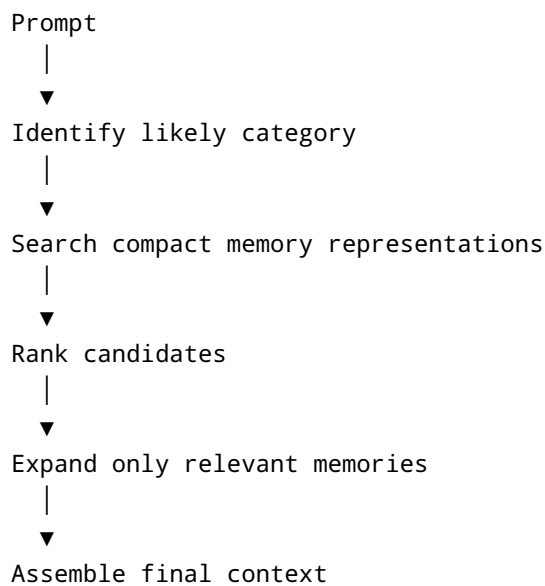
A naive architecture might conceptually operate like:



This can create several problems:

- Increasing retrieval cost
- Increasing context consumption
- Retrieval of irrelevant information
- Context pollution
- Redundant memories
- Difficulty scaling to very large memory stores
- Increased latency

SIMA proposes a progressive retrieval architecture:

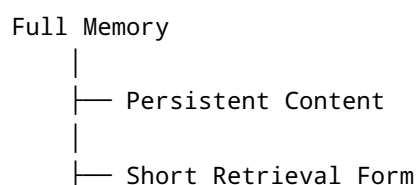


The architecture does **not** claim to increase the model's physical context window.

Instead, it aims to increase the **effective usable memory capacity** by reducing how much irrelevant information must be loaded into the active context.

Core Principle

SIMA separates the representation used to **store** a memory from the representation used to **find** that memory.



- Category
- Subcategory
- Entities
- Topics
- Embedding
- Relationships

The retrieval system searches the inexpensive representations first.

Full memory content is expanded only after relevance has been established.

Core principle

Store broadly. Retrieve narrowly. Expand progressively.

Memory Object

A SIMA memory record may contain:

```
{
  "memory_id": "mem_000001",
  "category": "ai_research",
  "subcategory": "memory_architecture",
  "retrieval_key": "SIMA",
  "short_form": "Category-indexed compressed memory retrieval architecture.",
  "entities": [
    "memory",
    "retrieval",
    "context",
    "indexing"
  ],
  "topics": [
    "AI",
    "memory",
    "retrieval",
    "architecture"
  ],
  "importance": 0.91,
```

```
{
  "confidence": 0.87,
  "timestamp": "2026-07-21T00:00:00Z",
  "source": "conversation",
  "full_content": "Full memory stored separately.",
  "embedding": "vector_reference",
  "relationships": [
    "mem_000014",
    "mem_000027"
  ]
}
```

The `short_form` is intended as a **retrieval representation**, not necessarily a human-readable summary.

The optimization target is:

Fast identification of relevance

rather than:

Maximum compression of information

Architecture

SIMA is composed of the following major components:

1. Query Interpreter

Converts a user query into a structured retrieval request.

Example:

"What was that memory architecture idea I had yesterday?"

May produce:

```
{
  "intent": "recall",
  "entities": [
    "memory architecture"
  ],
}
```

```
"time_reference": "recent",  
"requested_detail": "high"  
}
```

2. Category Router

Routes the query to relevant memory domains before semantic retrieval.

Example:

```
AI  
└─ Research  
   └─ Memory
```

This reduces the number of candidate memories requiring deeper evaluation.

3. Short-Form Index

Searches compact retrieval representations.

Example:

```
MEM_001  
Category-indexed compressed memory retrieval  
  
MEM_002  
Hierarchical memory routing  
  
MEM_003  
Graph-based associative memory  
  
MEM_004  
Long-context retrieval optimization
```

4. Relevance Ranker

Ranks candidate memories using multiple signals.

A conceptual scoring function is:

```
R(m,q) =  
  w_s * SemanticSimilarity  
+ w_c * CategoryMatch  
+ w_t * TemporalRelevance  
+ w_e * EntityOverlap  
+ w_i * Importance
```

The weights are experimental and should be benchmarked.

5. Memory Graph

Memories may be connected by explicit relationships.

```
MEM_001  
|  
├── related_to — MEM_004  
|  
├── derived_from — MEM_009  
|  
└── extends — MEM_021
```

This enables multi-hop retrieval.

6. Memory Expansion

Only high-confidence candidates are expanded into full memory objects.

This creates progressive retrieval:

```
Category  
  ↓  
Short Form  
  ↓  
Candidate  
  ↓  
Detailed Memory  
  ↓  
Full Memory
```

7. Context Assembler

Combines selected memories into the final context passed to the model.

The assembler should prioritize:

- Relevance
- Recency
- Confidence
- Importance
- Relationship coherence
- Token budget

Hierarchical Retrieval

SIMA supports multiple retrieval layers:

```
Level 4 – Category Index
      ↓
Level 3 – Entity / Keyword Index
      ↓
Level 2 – Short Retrieval Form
      ↓
Level 1 – Detailed Summary
      ↓
Level 0 – Full Memory
```

Retrieval proceeds from cheap representations to expensive representations.

The system should only descend deeper when additional information is required.

Adaptive Retrieval

SIMA can dynamically change retrieval depth based on confidence.

High confidence:

```
Query
  ↓
Category
  ↓
Top Memory
```


↓
Answer

Low confidence:

Query
↓
Category
↓
Broad Candidate Set
↓
Graph Traversal
↓
Expanded Memories
↓
Re-ranking
↓
Answer

The intended relationship is:

Retrieval Depth = $f(\text{Retrieval Confidence})$

Memory Graph

The memory graph is represented as:

$G = (V, E)$

where:

V = Memory Nodes
E = Relationships Between Memories

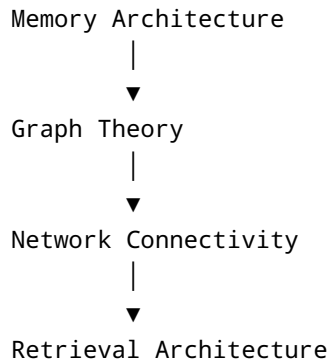
Possible relationship types include:

related_to
derived_from
extends

contradicts
supersedes
supports
references

A query may begin at one memory and traverse related memories.

Example:



This allows the system to retrieve conceptual relationships that may not be represented by a single embedding.

Expected Benefits

SIMA is designed to investigate whether indexed progressive retrieval can provide:

- Lower retrieval latency
- Lower token consumption
- Higher effective memory capacity
- Reduced context pollution
- Improved long-term recall
- Better multi-hop memory retrieval
- Improved scalability

These are **hypotheses**, not established results.

Benchmarking

SIMA should be compared against at least:

Baseline A

Flat vector retrieval.

Baseline B

Vector retrieval + reranking.

SIMA

Category routing + short-form retrieval + progressive expansion.

SIMA+

Category routing + short-form retrieval + graph traversal + adaptive expansion.

Measure:

```
Recall@K
Precision@K
Context Tokens
Retrieval Latency
Compute Cost
Long-Term Recall Accuracy
Multi-Hop Recall Accuracy
```

The primary hypothesis is:

```
SIMA

≈ or > Retrieval Quality

while reducing:

Tokens
Latency
Compute
```

Repository Structure

```
sima/
|
```

```
|— README.md
|— ARCHITECTURE.md
|— LICENSE
|— CONTRIBUTING.md
|
|— docs/
|   |— retrieval.md
|   |— memory-format.md
|   |— indexing.md
|   |— benchmarking.md
|
|— core/
|   |— memory_store.py
|   |— memory_index.py
|   |— category_router.py
|   |— relevance_ranker.py
|   |— graph_traversal.py
|   |— context_assembler.py
|
|— benchmarks/
|   |— baseline_vector.py
|   |— sima.py
|   |— evaluate.py
|
|— tests/
|
|— examples/
|   |— basic_retrieval.py
```

Research Questions

The project is explicitly open-ended.

Key questions include:

1. How much can retrieval latency be reduced?
2. How much can context consumption be reduced?
3. Does short-form indexing preserve recall?
4. How should short-form representations be generated?
5. What category hierarchy produces the best routing?
6. When should the system expand additional memories?
7. Does graph traversal improve multi-hop recall?
8. How does the architecture scale to millions or billions of memories?
9. How should conflicting or stale memories be handled?

10. Can memory compression improve effective context capacity without sacrificing accuracy?

Roadmap

Phase 1 — Prototype

- ☐ Memory schema
- ☐ Short-form generation
- ☐ Category indexing
- ☐ Basic retrieval
- ☐ Context assembly

Phase 2 — Benchmark

- ☐ Baseline vector retrieval
- ☐ SIMA retrieval
- ☐ Latency measurement
- ☐ Token measurement
- ☐ Recall measurement

Phase 3 — Advanced Retrieval

- ☐ Hierarchical indexes
- ☐ Adaptive expansion
- ☐ Memory graph
- ☐ Multi-hop retrieval
- ☐ Conflict resolution

Phase 4 — Scale

- ☐ Large memory corpus testing
 - ☐ Distributed indexes
 - ☐ Persistent graph storage
 - ☐ Production inference integration
-

Contributing

SIMA is an experimental open research project.

Contributions are welcome in:

- Retrieval algorithms
- Indexing
- Memory compression
- Graph-based retrieval

- Benchmarking
- Evaluation
- Scalability
- Formal analysis

The project prioritizes reproducible experiments over unverified performance claims.

License

To be determined.

Core Thesis

SIMA proposes that AI systems should separate the representation used to store memories from the representation used to retrieve them, using compact category-indexed representations to identify relevant memories before progressively expanding full memory content into the active context.

The objective is not to make the context window physically larger.

The objective is to make the system's **effective memory capacity** substantially larger without proportionally increasing the amount of information that must be actively loaded into context.